

SEABORN

Seaborn is a Python library for data visualization builds on top of the **matplotlib Python library**.

It means that you are working on python programming and want to plot some graphs then seaborn will help to do this task. It built using the matplotlib library use for the same.

It is specially used for statistical graphics. Means, you have some numeric types of data and want to analyze it using statistics then seaborn will help to draw data in a statistics way.

Seaborn Vs Matplotlib

If you know about matplotlib. It also uses for data visualization. So, in your mind, multiple questions will arrive and it's natural because of seaborn works the same.

But seaborn smatter than the matpotlib and plot graphs beautiful with less coding. You can do the same work with matplotlib but it is difficult because it has no such built-in function.

Python Seaborn Graphics

Using **seaborn data visualization** library plot below graphics.

- Line plot
- Histogram / Distplot
- Barplot
- Scatter Plot
- Heatmap
- Pailplot
- Facetgrid
- Boxplot
- Violin plot

How to Install Seaborn?

Follow the below command in terminal or command prompt if you have installed any **Python IDE** (Integrated Development Environment).

```
pip install seaborn
```

SEABORN

If you have installed **anaconda** then no need to **install seaborn** but while using it some error occur regarding seaborn installation then follow bellow command in anaconda prompt.

```
conda install seaborn
```

How to Import Seaborn?

It is easy to import the seaborn library if you have installed its dependency and any python IDE. Follow the bellow syntax in IDE.

```
1 import seaborn as sns
```

Note: Before importing seaborn import matplotlib because most of the matplotlib built-in function help to decorate it's graph. If you are loading dataset then also import numpy and pandas

Seaborn Line Plot

If you have two numeric variable datasets and worry about what relationship between them. Then Python **seaborn line plot** function will help to find it. Seaborn library provides **sns.lineplot()** function to draw a line graph of two numeric variables like x and y.

```
import seaborn as sns # for data visualization
import pandas as pd # for data analysis
import matplotlib.pyplot as plt # for data visualization
```

Seaborn provide **sns.lineplot()** function to draw beautiful single and multiple line plots using its parameters.

CREATIVE INSTITUTE DATA SCIENCE

SEABORN

```
import seaborn as sns # for data visualization
import pandas as pd # for data analysis
import matplotlib.pyplot as plt # for data visualization

days = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]
temperature = [36.6, 37, 37.7,39,40.1,43,43.4,45,45.6,40.1,44,45,46.8,47,47.8]

#create dataframe using two list days and temperature
temp_df = pd.DataFrame({"days":days, "temperature":temperature})

# Draw line plot
sns.lineplot(x = "days", y = "temperature", data=temp_df,)
plt.show() # to show graph
```

Load DataFrame from GitHub Seaborn Repository

github seaborn dataset # keyword

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

tips_df = sns.load_dataset("tips")
print(tips_df)
sns.lineplot(x='tip',y='total_bill',data=tips_df)
plt.show()
```

Draw line plot of tip and size

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

tips_df = sns.load_dataset("tips")
print(tips_df)

sns.lineplot(x = "tip", y = "size", data = tips_df)
plt.show()
```

SEABORN

Draw line plot of size and total_bill

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

tips_df = sns.load_dataset("tips")
print(tips_df)

sns.lineplot(x = "size", y = "total_bill", data = tips_df)
plt.show()
```

Seaborn Line Plot with Multiple Parameters

Drawn multiple line plot using **x**, **y** and **data** parameters. Now, we are using multiple parameters and see the amazing output.

hue => Get separate line plots for the third categorical variable. In the above graph draw relationship between **size (x-axis)** and total-bill (y-axis). Now, plotting separate line plots for Female and Male category of variable **sex**.

style => Give style to line plot, like dashes. Different for each line plot.

palette => Give **colormap** for graph. You can choose anyone from below which is separated by a comma.

dashes => If line plot with dashes then use **"False"** value for no dashes otherwise **"True"**.

markers => Give the markers for point like (x1,y1). for markers follow matplotlib line plot blog.

legend => Give legend. The default value is "brief" but you can give **"full"** or **"False"**. **False** for no legend.

SEABORN

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

tips_df = sns.load_dataset("tips")
print(tips_df)

sns.lineplot(x = "size", y = "total_bill", data = tips_df, hue = "sex",
             style = "sex", palette = "hot", dashes = False,
             markers = ["o", "<"], legend="brief",)

plt.title("Line Plot", fontsize = 20) # for title
plt.xlabel("Size", fontsize = 15) # label for x-axis
plt.ylabel("Total Bill", fontsize = 15) # label for y-axis
plt.show()
```

Note:

- We use only important parameters but you can use multiple depends on requirements.
- Seaborn line plot function support xlabel and ylabel but here we used separate functions to change its font size

multiple line plots

Using `sns.lineplot()` **hue** parameter, we can draw multiple line plot. In the above graphs drawn two line plots in a single graph (**Female** and **Male**) same way here use **day** categorical variable. Which have total 4-day categories?

CREATIVE INSTITUTE DATA SCIENCE

SEABORN

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

tips_df = sns.load_dataset("tips")
print(tips_df)

plt.figure(figsize = (16,9))
sns.set(style='darkgrid',)

# Draw line plot of size and total_bill with parameters and hue "day"
sns.lineplot(x = "size", y = "total_bill", data = tips_df, hue = "day",
             style = "day", palette = "hot", dashes = False,
             markers = ["o", "<", ">", "^"], legend="brief",)

plt.title("Line Plot", fontsize = 20)
plt.xlabel("Size", fontsize = 15)
plt.ylabel("Total Bill", fontsize = 15)
plt.show()
```